

main_notebook

November 30, 2025

1 Truth in Pixels: AI-Generated Image Detection

Team: Marston Ward, Jasper Dolar, Victor Salcedo

Course: AAI-521 Computer Vision

Date: November 2025

1.1 Project Overview

This notebook demonstrates a complete machine learning pipeline for detecting AI-generated images using deep learning. We compare three approaches:

1. **Custom CNN** - Baseline architecture built from scratch
2. **ResNet50** - Transfer learning with frozen pretrained backbone
3. **EfficientNet-B2** - Advanced transfer learning for high-resolution images

1.1.1 Dataset

- **Source:** Hugging Face (Hemg/AI-Generated-vs-Real-Images-Datasets)
- **Classes:** Real (0) vs AI-Generated (1)
- **Split:** 80% train, 10% validation, 10% test

1.1.2 Workflow

Data Preparation → Model Training → Evaluation → Comparison

1.2 1. Setup and Installation

Import required libraries and custom modules.

```
[29]: # Google Colab Setup - Clone repo and install dependencies
import sys
import os

# Check if running on Colab
IN_COLAB = 'google.colab' in sys.modules

if IN_COLAB:
```

```

print(" Setting up Google Colab environment...")

# Define repo name
repo_name = 'AAI-521-Computer-Vision-Image-Classification-Project'
repo_url = 'https://github.com/marstonsward/
↪AAI-521-Computer-Vision-Image-Classification-Project.git'

# Navigate to /content
os.chdir('/content')

# Clone or update repository
if os.path.exists(repo_name):
    print(f" Repository exists, pulling latest changes...")
    os.chdir(repo_name)
    !git reset --hard HEAD # Discard any local changes
    !git pull origin main
    print(" Repository updated")
else:
    print(f" Cloning repository...")
    !git clone {repo_url}
    os.chdir(repo_name)
    print(" Repository cloned")

print(f" Working directory: {os.getcwd()}")
print(f" Contents: {' '.join(os.listdir('.'))}")

# Install dependencies
!pip install -q datasets torch torchvision matplotlib seaborn scikit-learn
print(" Dependencies installed")

# Verify src directory exists
if os.path.exists('src'):
    print(" src/ directory found")
    print(f" src/ contains: {' '.join(os.listdir('src'))}")
else:
    print(" ERROR: src/ directory not found!")
    print(f" Available: {os.listdir('.')}")
else:
    print(" Running locally")

```

```

Setting up Google Colab environment...
Repository exists, pulling latest changes...
HEAD is now at 7367ce5 fix: Update execution count to null and reload EDA module
for batch visualization
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 5 (delta 3), reused 5 (delta 3), pack-reused 0 (from 0)

```

```

remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 5 (delta 3), reused 5 (delta 3), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 2.53 MiB | 11.67 MiB/s, done.
From https://github.com/marstonsward/AAI-521-Computer-Vision-Image-
Classification-Project
* branch          main          -> FETCH_HEAD
  7367ce5..55cdb20  main          -> origin/main
Updating 7367ce5..55cdb20
Fast-forward
  main_notebook.ipynb | 124
+++++-----
  src/eda.py           | 55 +++++
  2 files changed, 153 insertions(+), 26 deletions(-)
Unpacking objects: 100% (5/5), 2.53 MiB | 11.67 MiB/s, done.
From https://github.com/marstonsward/AAI-521-Computer-Vision-Image-
Classification-Project
* branch          main          -> FETCH_HEAD
  7367ce5..55cdb20  main          -> origin/main
Updating 7367ce5..55cdb20
Fast-forward
  main_notebook.ipynb | 124
+++++-----
  src/eda.py           | 55 +++++
  2 files changed, 153 insertions(+), 26 deletions(-)
Repository updated
Working directory: /content/AAI-521-Computer-Vision-Image-Classification-
Project
Contents: main_notebook.ipynb, src, report, requirements.txt, .gitignore,
.git, README.md
Repository updated
Working directory: /content/AAI-521-Computer-Vision-Image-Classification-
Project
Contents: main_notebook.ipynb, src, report, requirements.txt, .gitignore,
.git, README.md
Dependencies installed
src/ directory found
src/ contains: __init__.py, __pycache__, eda.py, models.py, training.py,
data.py, visualization.py
Dependencies installed
src/ directory found
src/ contains: __init__.py, __pycache__, eda.py, models.py, training.py,
data.py, visualization.py

```

```

[30]: # Standard library imports
import sys

```

```

from pathlib import Path
import random
import numpy as np
import matplotlib.pyplot as plt
import importlib

# Add project directory to Python path for module imports
project_root = Path.cwd()
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))
    print(f" Added to path: {project_root}")

# Verify we can see the src package
if (project_root / 'src').exists():
    print(f" src package accessible at: {project_root / 'src'}")
else:
    print(f" WARNING: src directory not found at {project_root}")

# PyTorch imports
import torch
import torch.nn as nn
import torch.optim as optim

# Custom modules - import and reload to get latest changes
from src import models, data, training, visualization, eda
importlib.reload(models)
importlib.reload(data)
importlib.reload(training)
importlib.reload(visualization)
importlib.reload(eda)

from src.models import CustomCNN, get_resnet50, get_efficientnet_b2,
    ↪count_parameters
from src.data import prepare_data, create_dataloaders
from src.training import Trainer, evaluate_model
from src.visualization import (plot_training_history, plot_confusion_matrix,
    compare_models, print_classification_report)
from src.eda import plot_class_distribution, visualize_samples

# Set random seeds
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

```

```
print(" All modules imported successfully")
```

```
src package accessible at: /content/AAI-521-Computer-Vision-Image-
Classification-Project/src
All modules imported successfully
```

1.3 2. Data Preparation

Load and split the dataset from Hugging Face.

```
[31]: %%time
# Setup device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f" Device: {device}")

# Load and split dataset
print("\n Loading dataset from Hugging Face...")
(train_images, train_labels,
 val_images, val_labels,
 test_images, test_labels) = prepare_data(seed=SEED)

print(f"\n Dataset Split:")
print(f" Training:  {len(train_images):,} images")
print(f" Validation: {len(val_images):,} images")
print(f" Test:      {len(test_images):,} images")
```

```
Device: cuda
```

```
Loading dataset from Hugging Face...
```

```
Downloading dataset 'Hemg/AI-Generated-vs-Real-Images-Datasets'...
```

```
First download: ~2-3 min, then cached for future runs
```

```
Dataset loaded: 152,710 total images
```

```
Preparing data splits...
```

```
Dataset loaded: 152,710 total images
```

```
Preparing data splits...
```

```
Dataset Split:
```

```
Training: 122,168 images
```

```
Validation: 15,271 images
```

```
Test: 15,271 images
```

```
CPU times: user 3min, sys: 1.54 s, total: 3min 2s
```

```
Wall time: 3min 3s
```

```
Dataset Split:
```

```
Training: 122,168 images
```

```
Validation: 15,271 images
```

```
Test: 15,271 images
```

```
CPU times: user 3min, sys: 1.54 s, total: 3min 2s
```

```
Wall time: 3min 3s
```

```
[32]: # Create DataLoaders
BATCH_SIZE = 32
NUM_WORKERS = 2

train_loader, val_loader, test_loader = create_dataloaders(
    train_images, train_labels,
    val_images, val_labels,
    test_images, test_labels,
    batch_size=BATCH_SIZE,
    num_workers=NUM_WORKERS
)

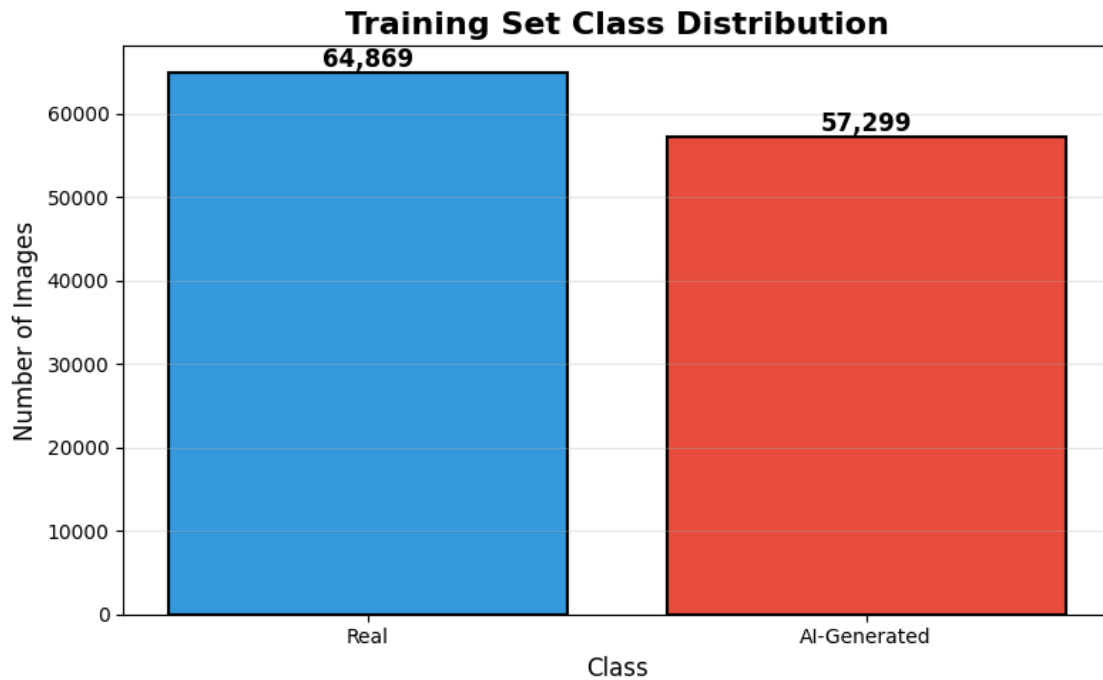
print(f" DataLoaders created:")
print(f"  Batch size: {BATCH_SIZE}")
print(f"  Train batches: {len(train_loader)}")
print(f"  Val batches:   {len(val_loader)}")
print(f"  Test batches:  {len(test_loader)}")
```

```
DataLoaders created:
Batch size: 32
Train batches: 3818
Val batches:   478
Test batches:  478
```

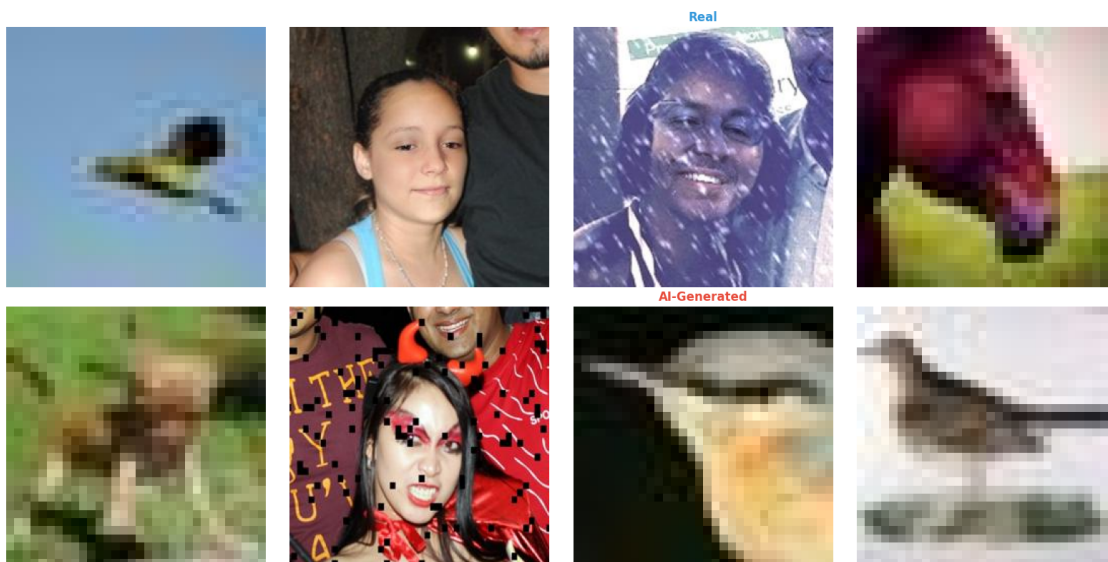
1.4 3. Exploratory Data Analysis

Visualize the dataset distribution and sample images.

```
[33]: # Class distribution using EDA module
plot_class_distribution(
    train_labels,
    class_names=['Real', 'AI-Generated'],
    title='Training Set Class Distribution'
)
```



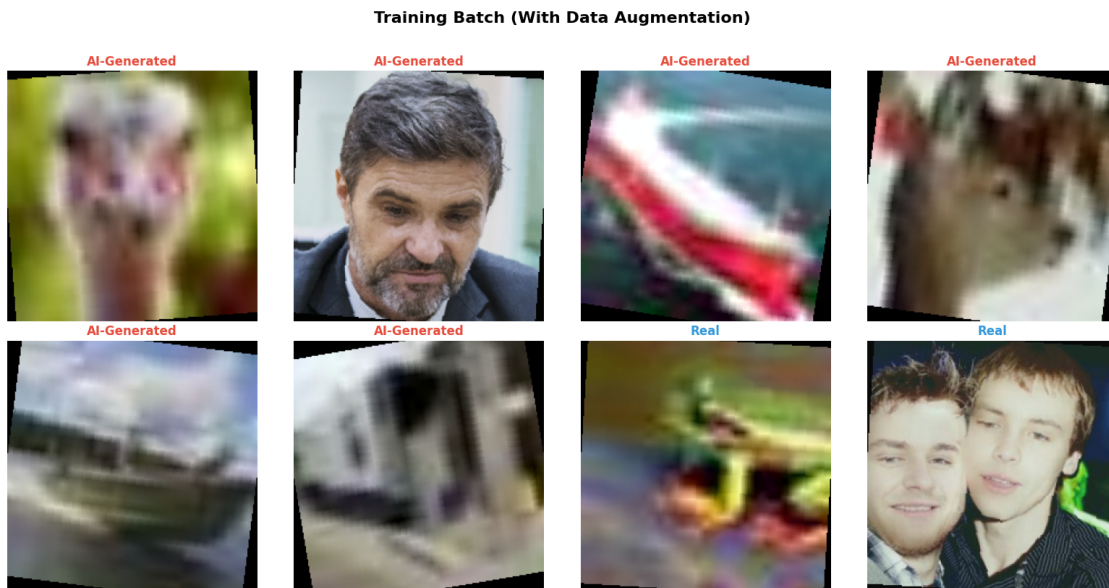
```
[34]: # Sample images using EDA module
visualize_samples(
    train_images,
    train_labels,
    class_names=['Real', 'AI-Generated'],
    n_per_class=4
)
```



```
[35]: # Visualize batch from training DataLoader (with augmentation)
from src.eda import visualize_batch
import importlib
import src.eda
importlib.reload(src.eda)
from src.eda import visualize_batch

print("Training batch samples (with data augmentation):")
visualize_batch(train_loader, class_names=['Real', 'AI-Generated'], n_images=8,
                title='Training Batch (With Data Augmentation)')
```

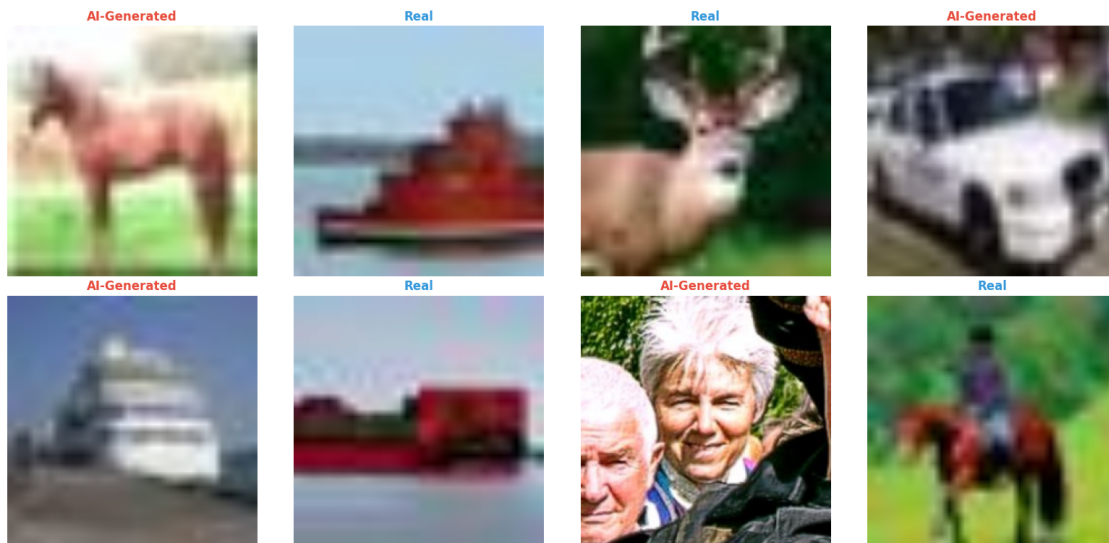
Training batch samples (with data augmentation):



```
[36]: # Visualize validation batch (without augmentation)
print("\nValidation batch samples (no augmentation):")
visualize_batch(val_loader, class_names=['Real', 'AI-Generated'], n_images=8,
                title='Validation Batch (No Augmentation - Only Resize & \u2192Normalize)')
```

Validation batch samples (no augmentation):

Validation Batch (No Augmentation - Only Resize & Normalize)



1.5 4. Model 1: Custom CNN

Baseline convolutional neural network with 3 conv layers, batch normalization, and dropout.

```
[37]: %%time
# Create model
cnn_model = CustomCNN(num_classes=1).to(device)
trainable, total = count_parameters(cnn_model)
print(f" CNN Parameters: {trainable:,} trainable / {total:,} total")

# Setup training
criterion_cnn = nn.BCEWithLogitsLoss()
optimizer_cnn = optim.Adam(cnn_model.parameters(), lr=0.001)

trainer_cnn = Trainer(cnn_model, device, criterion_cnn, optimizer_cnn,
    ↪use_amp=True)

# Train
print("\n Training Custom CNN...")
history_cnn = trainer_cnn.fit(train_loader, val_loader, num_epochs=10,
    save_path='models/cnn_best.pth')
```

CNN Parameters: 51,474,945 trainable / 51,474,945 total

Training Custom CNN...

Epoch 1/10

```
[ 381/3818] Loss: 3.3286 | Acc: 0.5605
[ 381/3818] Loss: 3.3286 | Acc: 0.5605
[ 762/3818] Loss: 1.9821 | Acc: 0.5932
[ 762/3818] Loss: 1.9821 | Acc: 0.5932
[1143/3818] Loss: 1.5289 | Acc: 0.6041
[1143/3818] Loss: 1.5289 | Acc: 0.6041
[1524/3818] Loss: 1.3016 | Acc: 0.6094
[1524/3818] Loss: 1.3016 | Acc: 0.6094
[1905/3818] Loss: 1.1639 | Acc: 0.6146
[1905/3818] Loss: 1.1639 | Acc: 0.6146
[2286/3818] Loss: 1.0713 | Acc: 0.6193
[2286/3818] Loss: 1.0713 | Acc: 0.6193
[2667/3818] Loss: 1.0044 | Acc: 0.6237
[2667/3818] Loss: 1.0044 | Acc: 0.6237
[3048/3818] Loss: 0.9534 | Acc: 0.6279
[3048/3818] Loss: 0.9534 | Acc: 0.6279
[3429/3818] Loss: 0.9143 | Acc: 0.6307
[3429/3818] Loss: 0.9143 | Acc: 0.6307
[3810/3818] Loss: 0.8825 | Acc: 0.6333
[3810/3818] Loss: 0.8825 | Acc: 0.6333
Train Loss: 0.8820 | Train Acc: 0.6333
Val   Loss: 0.5433 | Val   Acc: 0.7325
Train Loss: 0.8820 | Train Acc: 0.6333
Val   Loss: 0.5433 | Val   Acc: 0.7325
  Saved best model (val_acc: 0.7325)
```

Epoch 2/10

 Saved best model (val_acc: 0.7325)

Epoch 2/10

[381/3818] Loss: 0.5876 | Acc: 0.6681
[381/3818] Loss: 0.5876 | Acc: 0.6681
[762/3818] Loss: 0.5844 | Acc: 0.6755
[762/3818] Loss: 0.5844 | Acc: 0.6755
[1143/3818] Loss: 0.5816 | Acc: 0.6789
[1143/3818] Loss: 0.5816 | Acc: 0.6789
[1524/3818] Loss: 0.5817 | Acc: 0.6799
[1524/3818] Loss: 0.5817 | Acc: 0.6799
[1905/3818] Loss: 0.5800 | Acc: 0.6811
[1905/3818] Loss: 0.5800 | Acc: 0.6811
[2286/3818] Loss: 0.5783 | Acc: 0.6822
[2286/3818] Loss: 0.5783 | Acc: 0.6822
[2667/3818] Loss: 0.5765 | Acc: 0.6855
[2667/3818] Loss: 0.5765 | Acc: 0.6855
[3048/3818] Loss: 0.5735 | Acc: 0.6888
[3048/3818] Loss: 0.5735 | Acc: 0.6888

```
[3429/3818] Loss: 0.5711 | Acc: 0.6912
[3429/3818] Loss: 0.5711 | Acc: 0.6912
[3810/3818] Loss: 0.5692 | Acc: 0.6933
[3810/3818] Loss: 0.5692 | Acc: 0.6933
Train Loss: 0.5692 | Train Acc: 0.6934
Val   Loss: 0.4925 | Val   Acc: 0.7961
Train Loss: 0.5692 | Train Acc: 0.6934
Val   Loss: 0.4925 | Val   Acc: 0.7961
  Saved best model (val_acc: 0.7961)
```

Epoch 3/10

```
-----
  Saved best model (val_acc: 0.7961)
```

Epoch 3/10

```
-----
[ 381/3818] Loss: 0.5417 | Acc: 0.7174
[ 381/3818] Loss: 0.5417 | Acc: 0.7174
[ 762/3818] Loss: 0.5422 | Acc: 0.7196
[ 762/3818] Loss: 0.5422 | Acc: 0.7196
[1143/3818] Loss: 0.5408 | Acc: 0.7203
[1143/3818] Loss: 0.5408 | Acc: 0.7203
[1524/3818] Loss: 0.5410 | Acc: 0.7205
[1524/3818] Loss: 0.5410 | Acc: 0.7205
[1905/3818] Loss: 0.5382 | Acc: 0.7231
[1905/3818] Loss: 0.5382 | Acc: 0.7231
[2286/3818] Loss: 0.5370 | Acc: 0.7248
[2286/3818] Loss: 0.5370 | Acc: 0.7248
[2667/3818] Loss: 0.5355 | Acc: 0.7259
[2667/3818] Loss: 0.5355 | Acc: 0.7259
[3048/3818] Loss: 0.5351 | Acc: 0.7262
[3048/3818] Loss: 0.5351 | Acc: 0.7262
[3429/3818] Loss: 0.5338 | Acc: 0.7272
[3429/3818] Loss: 0.5338 | Acc: 0.7272
[3810/3818] Loss: 0.5328 | Acc: 0.7277
[3810/3818] Loss: 0.5328 | Acc: 0.7277
Train Loss: 0.5328 | Train Acc: 0.7277
Val   Loss: 0.4250 | Val   Acc: 0.8394
Train Loss: 0.5328 | Train Acc: 0.7277
Val   Loss: 0.4250 | Val   Acc: 0.8394
  Saved best model (val_acc: 0.8394)
```

Epoch 4/10

```
-----
  Saved best model (val_acc: 0.8394)
```

Epoch 4/10

```

[ 381/3818] Loss: 0.5161 | Acc: 0.7426
[ 381/3818] Loss: 0.5161 | Acc: 0.7426
[ 762/3818] Loss: 0.5187 | Acc: 0.7384
[ 762/3818] Loss: 0.5187 | Acc: 0.7384
[1143/3818] Loss: 0.5163 | Acc: 0.7398
[1143/3818] Loss: 0.5163 | Acc: 0.7398
[1524/3818] Loss: 0.5157 | Acc: 0.7413
[1524/3818] Loss: 0.5157 | Acc: 0.7413
[1905/3818] Loss: 0.5162 | Acc: 0.7415
[1905/3818] Loss: 0.5162 | Acc: 0.7415
[2286/3818] Loss: 0.5161 | Acc: 0.7418
[2286/3818] Loss: 0.5161 | Acc: 0.7418
[2667/3818] Loss: 0.5150 | Acc: 0.7431
[2667/3818] Loss: 0.5150 | Acc: 0.7431
[3048/3818] Loss: 0.5143 | Acc: 0.7436
[3048/3818] Loss: 0.5143 | Acc: 0.7436
[3429/3818] Loss: 0.5136 | Acc: 0.7442
[3429/3818] Loss: 0.5136 | Acc: 0.7442
[3810/3818] Loss: 0.5133 | Acc: 0.7447
[3810/3818] Loss: 0.5133 | Acc: 0.7447
Train Loss: 0.5132 | Train Acc: 0.7447
Val   Loss: 0.4376 | Val   Acc: 0.8180

```

Epoch 5/10

```

-----
Train Loss: 0.5132 | Train Acc: 0.7447
Val   Loss: 0.4376 | Val   Acc: 0.8180

```

Epoch 5/10

```

-----
[ 381/3818] Loss: 0.5055 | Acc: 0.7460
[ 381/3818] Loss: 0.5055 | Acc: 0.7460
[ 762/3818] Loss: 0.5039 | Acc: 0.7502
[ 762/3818] Loss: 0.5039 | Acc: 0.7502
[1143/3818] Loss: 0.5033 | Acc: 0.7505
[1143/3818] Loss: 0.5033 | Acc: 0.7505
[1524/3818] Loss: 0.5022 | Acc: 0.7522
[1524/3818] Loss: 0.5022 | Acc: 0.7522
[1905/3818] Loss: 0.5013 | Acc: 0.7529
[1905/3818] Loss: 0.5013 | Acc: 0.7529
[2286/3818] Loss: 0.5007 | Acc: 0.7540
[2286/3818] Loss: 0.5007 | Acc: 0.7540
[2667/3818] Loss: 0.4998 | Acc: 0.7541
[2667/3818] Loss: 0.4998 | Acc: 0.7541
[3048/3818] Loss: 0.4990 | Acc: 0.7551
[3048/3818] Loss: 0.4990 | Acc: 0.7551
[3429/3818] Loss: 0.4984 | Acc: 0.7556
[3429/3818] Loss: 0.4984 | Acc: 0.7556

```

[3810/3818] Loss: 0.4976 | Acc: 0.7564
[3810/3818] Loss: 0.4976 | Acc: 0.7564
Train Loss: 0.4977 | Train Acc: 0.7564
Val Loss: 0.4359 | Val Acc: 0.8255

Epoch 6/10

Train Loss: 0.4977 | Train Acc: 0.7564
Val Loss: 0.4359 | Val Acc: 0.8255

Epoch 6/10

[381/3818] Loss: 0.4907 | Acc: 0.7623
[381/3818] Loss: 0.4907 | Acc: 0.7623
[762/3818] Loss: 0.4887 | Acc: 0.7639
[762/3818] Loss: 0.4887 | Acc: 0.7639
[1143/3818] Loss: 0.4887 | Acc: 0.7652
[1143/3818] Loss: 0.4887 | Acc: 0.7652
[1524/3818] Loss: 0.4921 | Acc: 0.7625
[1524/3818] Loss: 0.4921 | Acc: 0.7625
[1905/3818] Loss: 0.4928 | Acc: 0.7605
[1905/3818] Loss: 0.4928 | Acc: 0.7605
[2286/3818] Loss: 0.4913 | Acc: 0.7615
[2286/3818] Loss: 0.4913 | Acc: 0.7615
[2667/3818] Loss: 0.4907 | Acc: 0.7610
[2667/3818] Loss: 0.4907 | Acc: 0.7610
[3048/3818] Loss: 0.4904 | Acc: 0.7614
[3048/3818] Loss: 0.4904 | Acc: 0.7614
[3429/3818] Loss: 0.4891 | Acc: 0.7622
[3429/3818] Loss: 0.4891 | Acc: 0.7622
[3810/3818] Loss: 0.4885 | Acc: 0.7625
[3810/3818] Loss: 0.4885 | Acc: 0.7625
Train Loss: 0.4885 | Train Acc: 0.7626
Val Loss: 0.3678 | Val Acc: 0.8741
Train Loss: 0.4885 | Train Acc: 0.7626
Val Loss: 0.3678 | Val Acc: 0.8741
Saved best model (val_acc: 0.8741)

Epoch 7/10

Saved best model (val_acc: 0.8741)

Epoch 7/10

[381/3818] Loss: 0.4886 | Acc: 0.7616
[381/3818] Loss: 0.4886 | Acc: 0.7616
[762/3818] Loss: 0.4886 | Acc: 0.7615
[762/3818] Loss: 0.4886 | Acc: 0.7615

```
[1143/3818] Loss: 0.4849 | Acc: 0.7646
[1143/3818] Loss: 0.4849 | Acc: 0.7646
[1524/3818] Loss: 0.4849 | Acc: 0.7651
[1524/3818] Loss: 0.4849 | Acc: 0.7651
[1905/3818] Loss: 0.4846 | Acc: 0.7657
[1905/3818] Loss: 0.4846 | Acc: 0.7657
[2286/3818] Loss: 0.4815 | Acc: 0.7681
[2286/3818] Loss: 0.4815 | Acc: 0.7681
[2667/3818] Loss: 0.4816 | Acc: 0.7684
[2667/3818] Loss: 0.4816 | Acc: 0.7684
[3048/3818] Loss: 0.4809 | Acc: 0.7690
[3048/3818] Loss: 0.4809 | Acc: 0.7690
[3429/3818] Loss: 0.4804 | Acc: 0.7692
[3429/3818] Loss: 0.4804 | Acc: 0.7692
[3810/3818] Loss: 0.4796 | Acc: 0.7694
[3810/3818] Loss: 0.4796 | Acc: 0.7694
Train Loss: 0.4796 | Train Acc: 0.7694
Val   Loss: 0.3503 | Val   Acc: 0.8794
Train Loss: 0.4796 | Train Acc: 0.7694
Val   Loss: 0.3503 | Val   Acc: 0.8794
  Saved best model (val_acc: 0.8794)
```

Epoch 8/10

 Saved best model (val_acc: 0.8794)

Epoch 8/10

[381/3818] Loss: 0.4700 | Acc: 0.7779
[381/3818] Loss: 0.4700 | Acc: 0.7779
[762/3818] Loss: 0.4690 | Acc: 0.7787
[762/3818] Loss: 0.4690 | Acc: 0.7787
[1143/3818] Loss: 0.4706 | Acc: 0.7770
[1143/3818] Loss: 0.4706 | Acc: 0.7770
[1524/3818] Loss: 0.4716 | Acc: 0.7763
[1524/3818] Loss: 0.4716 | Acc: 0.7763
[1905/3818] Loss: 0.4720 | Acc: 0.7759
[1905/3818] Loss: 0.4720 | Acc: 0.7759
[2286/3818] Loss: 0.4704 | Acc: 0.7770
[2286/3818] Loss: 0.4704 | Acc: 0.7770
[2667/3818] Loss: 0.4709 | Acc: 0.7761
[2667/3818] Loss: 0.4709 | Acc: 0.7761
[3048/3818] Loss: 0.4709 | Acc: 0.7763
[3048/3818] Loss: 0.4709 | Acc: 0.7763
[3429/3818] Loss: 0.4710 | Acc: 0.7760
[3429/3818] Loss: 0.4710 | Acc: 0.7760
[3810/3818] Loss: 0.4712 | Acc: 0.7757
[3810/3818] Loss: 0.4712 | Acc: 0.7757

Train Loss: 0.4713 | Train Acc: 0.7757
Val Loss: 0.3564 | Val Acc: 0.8815
Train Loss: 0.4713 | Train Acc: 0.7757
Val Loss: 0.3564 | Val Acc: 0.8815
Saved best model (val_acc: 0.8815)

Epoch 9/10

Saved best model (val_acc: 0.8815)

Epoch 9/10

[381/3818] Loss: 0.4646 | Acc: 0.7798
[381/3818] Loss: 0.4646 | Acc: 0.7798
[762/3818] Loss: 0.4628 | Acc: 0.7807
[762/3818] Loss: 0.4628 | Acc: 0.7807
[1143/3818] Loss: 0.4619 | Acc: 0.7816
[1143/3818] Loss: 0.4619 | Acc: 0.7816
[1524/3818] Loss: 0.4611 | Acc: 0.7831
[1524/3818] Loss: 0.4611 | Acc: 0.7831
[1905/3818] Loss: 0.4620 | Acc: 0.7820
[1905/3818] Loss: 0.4620 | Acc: 0.7820
[2286/3818] Loss: 0.4621 | Acc: 0.7819
[2286/3818] Loss: 0.4621 | Acc: 0.7819
[2667/3818] Loss: 0.4626 | Acc: 0.7816
[2667/3818] Loss: 0.4626 | Acc: 0.7816
[3048/3818] Loss: 0.4634 | Acc: 0.7805
[3048/3818] Loss: 0.4634 | Acc: 0.7805
[3429/3818] Loss: 0.4634 | Acc: 0.7808
[3429/3818] Loss: 0.4634 | Acc: 0.7808
[3810/3818] Loss: 0.4629 | Acc: 0.7810
[3810/3818] Loss: 0.4629 | Acc: 0.7810
Train Loss: 0.4629 | Train Acc: 0.7810
Val Loss: 0.3282 | Val Acc: 0.8935
Train Loss: 0.4629 | Train Acc: 0.7810
Val Loss: 0.3282 | Val Acc: 0.8935
Saved best model (val_acc: 0.8935)

Epoch 10/10

Saved best model (val_acc: 0.8935)

Epoch 10/10

[381/3818] Loss: 0.4641 | Acc: 0.7773
[381/3818] Loss: 0.4641 | Acc: 0.7773
[762/3818] Loss: 0.4597 | Acc: 0.7832
[762/3818] Loss: 0.4597 | Acc: 0.7832

```

[1143/3818] Loss: 0.4584 | Acc: 0.7846
[1143/3818] Loss: 0.4584 | Acc: 0.7846
[1524/3818] Loss: 0.4582 | Acc: 0.7850
[1524/3818] Loss: 0.4582 | Acc: 0.7850
[1905/3818] Loss: 0.4532 | Acc: 0.7893
[1905/3818] Loss: 0.4532 | Acc: 0.7893
[2286/3818] Loss: 0.4432 | Acc: 0.7979
[2286/3818] Loss: 0.4432 | Acc: 0.7979
[2667/3818] Loss: 0.4369 | Acc: 0.8039
[2667/3818] Loss: 0.4369 | Acc: 0.8039
[3048/3818] Loss: 0.4314 | Acc: 0.8081
[3048/3818] Loss: 0.4314 | Acc: 0.8081
[3429/3818] Loss: 0.4241 | Acc: 0.8133
[3429/3818] Loss: 0.4241 | Acc: 0.8133
[3810/3818] Loss: 0.4189 | Acc: 0.8173
[3810/3818] Loss: 0.4189 | Acc: 0.8173
Train Loss: 0.4187 | Train Acc: 0.8174
Val   Loss: 0.2794 | Val   Acc: 0.8991
Train Loss: 0.4187 | Train Acc: 0.8174
Val   Loss: 0.2794 | Val   Acc: 0.8991
  Saved best model (val_acc: 0.8991)

  Training complete in 5718s
Best validation accuracy: 0.8991
CPU times: user 17min 3s, sys: 3min 20s, total: 20min 23s
Wall time: 1h 35min 18s
  Saved best model (val_acc: 0.8991)

  Training complete in 5718s
Best validation accuracy: 0.8991
CPU times: user 17min 3s, sys: 3min 20s, total: 20min 23s
Wall time: 1h 35min 18s

```

```

[38]: # Plot training history
plot_training_history(history_cnn, "Custom CNN Training History")

# Evaluate on test set (convert binary to 2-class for consistency)
# For CNN with BCEWithLogitsLoss, we need special handling
cnn_model.eval()
all_preds_cnn = []
all_labels_cnn = []
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = cnn_model(images)
        preds = (torch.sigmoid(outputs) > 0.5).long().squeeze()
        all_preds_cnn.extend(preds.cpu().numpy())

```

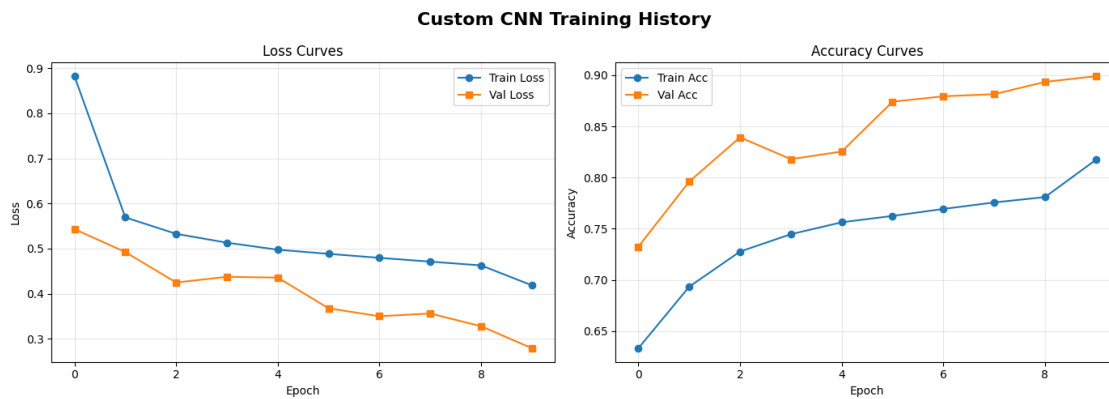
```

all_labels_cnn.extend(labels.cpu().numpy())

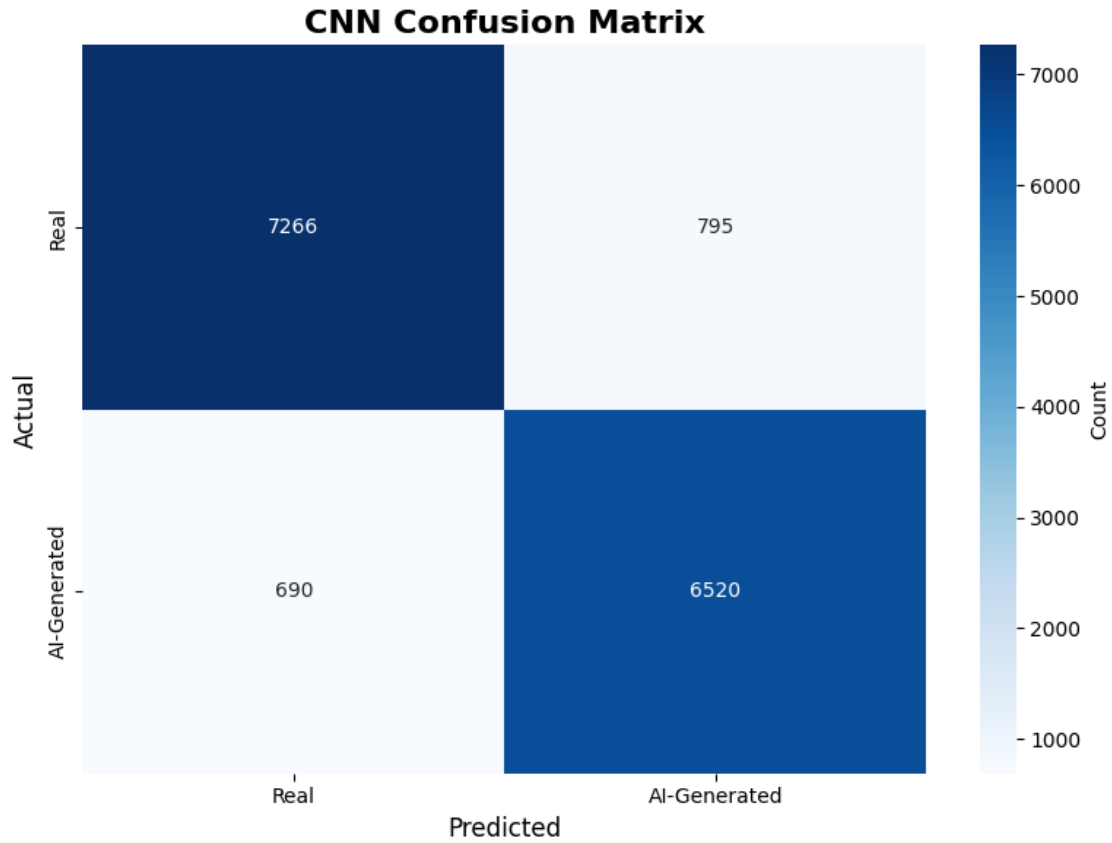
test_acc_cnn = np.mean(np.array(all_preds_cnn) == np.array(all_labels_cnn))
print(f"\n Test Accuracy: {test_acc_cnn:.4f}")

plot_confusion_matrix(all_labels_cnn, all_preds_cnn, title='CNN Confusion_
↪Matrix')
print_classification_report(all_labels_cnn, all_preds_cnn)

```



Test Accuracy: 0.9028



```
=====
CLASSIFICATION REPORT
=====
```

	precision	recall	f1-score	support
Real	0.91	0.90	0.91	8061
AI-Generated	0.89	0.90	0.90	7210
accuracy			0.90	15271
macro avg	0.90	0.90	0.90	15271
weighted avg	0.90	0.90	0.90	15271

	precision	recall	f1-score	support
Real	0.91	0.90	0.91	8061
AI-Generated	0.89	0.90	0.90	7210
accuracy			0.90	15271
macro avg	0.90	0.90	0.90	15271
weighted avg	0.90	0.90	0.90	15271

1.6 5. Model 2: ResNet50 Transfer Learning

Pretrained ResNet50 with frozen backbone and custom classifier.

```
[39]: %%time
# Create model
resnet_model = get_resnet50(num_classes=2, freeze_backbone=True).to(device)
trainable, total = count_parameters(resnet_model)
print(f" ResNet50 Parameters: {trainable:,} trainable / {total:,} total")

# Setup training
criterion_resnet = nn.CrossEntropyLoss()
optimizer_resnet = optim.Adam(resnet_model.fc.parameters(), lr=1e-4,
    ↪weight_decay=1e-4)

trainer_resnet = Trainer(resnet_model, device, criterion_resnet,
    ↪optimizer_resnet, use_amp=True)

# Train
print("\n Training ResNet50...")
history_resnet = trainer_resnet.fit(train_loader, val_loader, num_epochs=5,
    save_path='models/resnet50_best.pth')
```

Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to
/root/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth

100%| | 97.8M/97.8M [00:00<00:00, 236MB/s]

ResNet50 Parameters: 4,098 trainable / 23,512,130 total

Training ResNet50...

Epoch 1/5

```
-----
[ 381/3818] Loss: 0.6165 | Acc: 0.6611
[ 381/3818] Loss: 0.6165 | Acc: 0.6611
[ 762/3818] Loss: 0.5756 | Acc: 0.7025
[ 762/3818] Loss: 0.5756 | Acc: 0.7025
[1143/3818] Loss: 0.5528 | Acc: 0.7211
[1143/3818] Loss: 0.5528 | Acc: 0.7211
[1524/3818] Loss: 0.5360 | Acc: 0.7337
[1524/3818] Loss: 0.5360 | Acc: 0.7337
[1905/3818] Loss: 0.5245 | Acc: 0.7418
[1905/3818] Loss: 0.5245 | Acc: 0.7418
[2286/3818] Loss: 0.5160 | Acc: 0.7477
[2286/3818] Loss: 0.5160 | Acc: 0.7477
```

```
[2667/3818] Loss: 0.5079 | Acc: 0.7534
[2667/3818] Loss: 0.5079 | Acc: 0.7534
[3048/3818] Loss: 0.5016 | Acc: 0.7578
[3048/3818] Loss: 0.5016 | Acc: 0.7578
[3429/3818] Loss: 0.4958 | Acc: 0.7621
[3429/3818] Loss: 0.4958 | Acc: 0.7621
[3810/3818] Loss: 0.4914 | Acc: 0.7645
[3810/3818] Loss: 0.4914 | Acc: 0.7645
Train Loss: 0.4913 | Train Acc: 0.7646
Val   Loss: 0.4752 | Val   Acc: 0.7656
  Saved best model (val_acc: 0.7656)
```

Epoch 2/5

```
-----
Train Loss: 0.4913 | Train Acc: 0.7646
Val   Loss: 0.4752 | Val   Acc: 0.7656
  Saved best model (val_acc: 0.7656)
```

Epoch 2/5

```
-----
[ 381/3818] Loss: 0.4470 | Acc: 0.7920
[ 381/3818] Loss: 0.4470 | Acc: 0.7920
[ 762/3818] Loss: 0.4422 | Acc: 0.7953
[ 762/3818] Loss: 0.4422 | Acc: 0.7953
[1143/3818] Loss: 0.4405 | Acc: 0.7969
[1143/3818] Loss: 0.4405 | Acc: 0.7969
[1524/3818] Loss: 0.4403 | Acc: 0.7964
[1524/3818] Loss: 0.4403 | Acc: 0.7964
[1905/3818] Loss: 0.4404 | Acc: 0.7968
[1905/3818] Loss: 0.4404 | Acc: 0.7968
[2286/3818] Loss: 0.4400 | Acc: 0.7969
[2286/3818] Loss: 0.4400 | Acc: 0.7969
[2667/3818] Loss: 0.4380 | Acc: 0.7982
[2667/3818] Loss: 0.4380 | Acc: 0.7982
[3048/3818] Loss: 0.4372 | Acc: 0.7984
[3048/3818] Loss: 0.4372 | Acc: 0.7984
[3429/3818] Loss: 0.4362 | Acc: 0.7986
[3429/3818] Loss: 0.4362 | Acc: 0.7986
[3810/3818] Loss: 0.4356 | Acc: 0.7988
[3810/3818] Loss: 0.4356 | Acc: 0.7988
Train Loss: 0.4357 | Train Acc: 0.7987
Val   Loss: 0.4536 | Val   Acc: 0.7817
Train Loss: 0.4357 | Train Acc: 0.7987
Val   Loss: 0.4536 | Val   Acc: 0.7817
  Saved best model (val_acc: 0.7817)
```

Epoch 3/5

Saved best model (val_acc: 0.7817)

Epoch 3/5

```
-----  
[ 381/3818] Loss: 0.4301 | Acc: 0.8006  
[ 381/3818] Loss: 0.4301 | Acc: 0.8006  
[ 762/3818] Loss: 0.4335 | Acc: 0.7976  
[ 762/3818] Loss: 0.4335 | Acc: 0.7976  
[1143/3818] Loss: 0.4274 | Acc: 0.8007  
[1143/3818] Loss: 0.4274 | Acc: 0.8007  
[1524/3818] Loss: 0.4260 | Acc: 0.8018  
[1524/3818] Loss: 0.4260 | Acc: 0.8018  
[1905/3818] Loss: 0.4249 | Acc: 0.8026  
[1905/3818] Loss: 0.4249 | Acc: 0.8026  
[2286/3818] Loss: 0.4244 | Acc: 0.8031  
[2286/3818] Loss: 0.4244 | Acc: 0.8031  
[2667/3818] Loss: 0.4241 | Acc: 0.8037  
[2667/3818] Loss: 0.4241 | Acc: 0.8037  
[3048/3818] Loss: 0.4237 | Acc: 0.8043  
[3048/3818] Loss: 0.4237 | Acc: 0.8043  
[3429/3818] Loss: 0.4240 | Acc: 0.8041  
[3429/3818] Loss: 0.4240 | Acc: 0.8041  
[3810/3818] Loss: 0.4236 | Acc: 0.8044  
[3810/3818] Loss: 0.4236 | Acc: 0.8044  
Train Loss: 0.4236 | Train Acc: 0.8044  
Val   Loss: 0.4993 | Val   Acc: 0.7540
```

Epoch 4/5

```
-----  
Train Loss: 0.4236 | Train Acc: 0.8044  
Val   Loss: 0.4993 | Val   Acc: 0.7540
```

Epoch 4/5

```
-----  
[ 381/3818] Loss: 0.4226 | Acc: 0.8083  
[ 381/3818] Loss: 0.4226 | Acc: 0.8083  
[ 762/3818] Loss: 0.4213 | Acc: 0.8095  
[ 762/3818] Loss: 0.4213 | Acc: 0.8095  
[1143/3818] Loss: 0.4193 | Acc: 0.8088  
[1143/3818] Loss: 0.4193 | Acc: 0.8088  
[1524/3818] Loss: 0.4181 | Acc: 0.8091  
[1524/3818] Loss: 0.4181 | Acc: 0.8091  
[1905/3818] Loss: 0.4181 | Acc: 0.8090  
[1905/3818] Loss: 0.4181 | Acc: 0.8090  
[2286/3818] Loss: 0.4174 | Acc: 0.8095  
[2286/3818] Loss: 0.4174 | Acc: 0.8095  
[2667/3818] Loss: 0.4165 | Acc: 0.8101  
[2667/3818] Loss: 0.4165 | Acc: 0.8101
```

```
[3048/3818] Loss: 0.4154 | Acc: 0.8102
[3048/3818] Loss: 0.4154 | Acc: 0.8102
[3429/3818] Loss: 0.4157 | Acc: 0.8101
[3429/3818] Loss: 0.4157 | Acc: 0.8101
[3810/3818] Loss: 0.4147 | Acc: 0.8105
[3810/3818] Loss: 0.4147 | Acc: 0.8105
Train Loss: 0.4147 | Train Acc: 0.8105
Val   Loss: 0.4342 | Val   Acc: 0.7924
Train Loss: 0.4147 | Train Acc: 0.8105
Val   Loss: 0.4342 | Val   Acc: 0.7924
  Saved best model (val_acc: 0.7924)
```

Epoch 5/5

 Saved best model (val_acc: 0.7924)

Epoch 5/5

[381/3818] Loss: 0.4066 | Acc: 0.8147
[381/3818] Loss: 0.4066 | Acc: 0.8147
[762/3818] Loss: 0.4081 | Acc: 0.8133
[762/3818] Loss: 0.4081 | Acc: 0.8133
[1143/3818] Loss: 0.4060 | Acc: 0.8147
[1143/3818] Loss: 0.4060 | Acc: 0.8147
[1524/3818] Loss: 0.4057 | Acc: 0.8145
[1524/3818] Loss: 0.4057 | Acc: 0.8145
[1905/3818] Loss: 0.4064 | Acc: 0.8145
[1905/3818] Loss: 0.4064 | Acc: 0.8145
[2286/3818] Loss: 0.4068 | Acc: 0.8152
[2286/3818] Loss: 0.4068 | Acc: 0.8152
[2667/3818] Loss: 0.4073 | Acc: 0.8146
[2667/3818] Loss: 0.4073 | Acc: 0.8146
[3048/3818] Loss: 0.4075 | Acc: 0.8147
[3048/3818] Loss: 0.4075 | Acc: 0.8147
[3429/3818] Loss: 0.4078 | Acc: 0.8143
[3429/3818] Loss: 0.4078 | Acc: 0.8143
[3810/3818] Loss: 0.4075 | Acc: 0.8142
[3810/3818] Loss: 0.4075 | Acc: 0.8142
Train Loss: 0.4074 | Train Acc: 0.8142
Val Loss: 0.4166 | Val Acc: 0.8063
Train Loss: 0.4074 | Train Acc: 0.8142
Val Loss: 0.4166 | Val Acc: 0.8063
 Saved best model (val_acc: 0.8063)

Training complete in 2837s
Best validation accuracy: 0.8063
CPU times: user 6min 28s, sys: 1min 40s, total: 8min 8s
Wall time: 47min 18s

Saved best model (val_acc: 0.8063)

Training complete in 2837s

Best validation accuracy: 0.8063

CPU times: user 6min 28s, sys: 1min 40s, total: 8min 8s

Wall time: 47min 18s

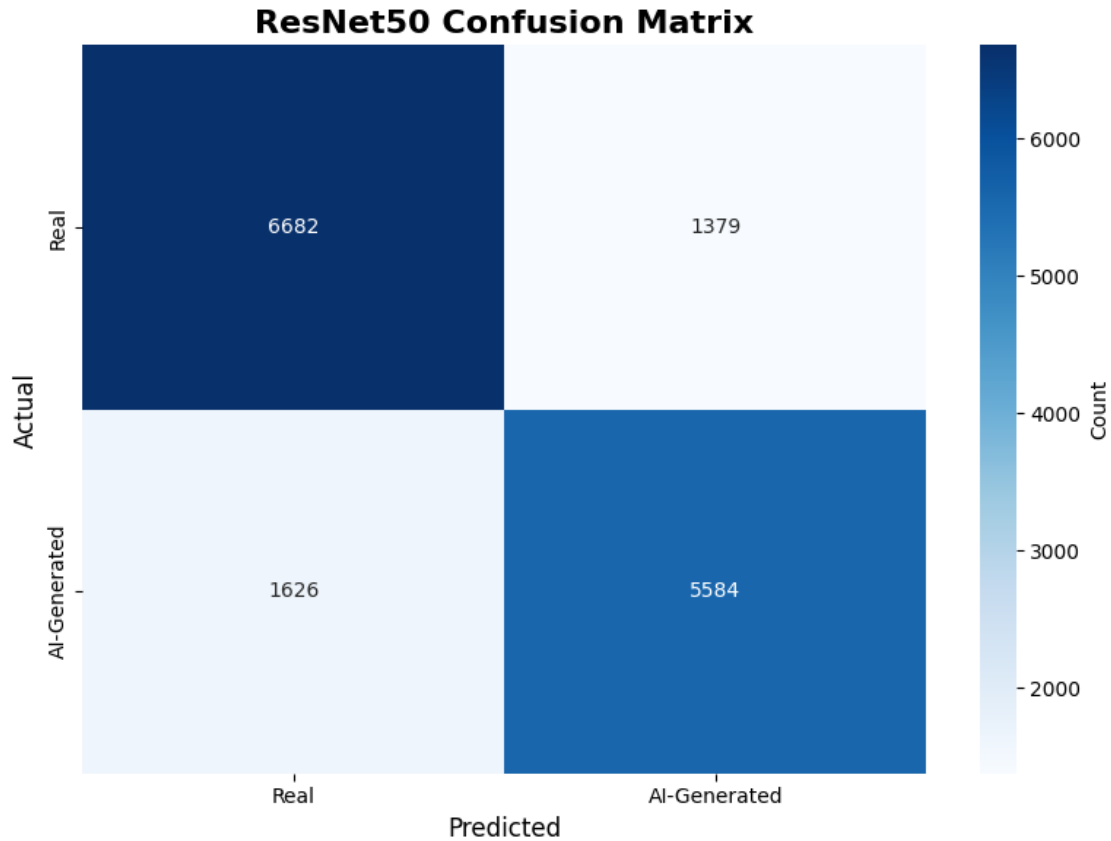
```
[40]: # Plot and evaluate
plot_training_history(history_resnet, "ResNet50 Training History")

test_acc_resnet, all_preds_resnet, all_labels_resnet = evaluate_model(resnet_model, test_loader, device)
print(f"\n Test Accuracy: {test_acc_resnet:.4f}")

plot_confusion_matrix(all_labels_resnet, all_preds_resnet, title='ResNet50 Confusion Matrix')
print_classification_report(all_labels_resnet, all_preds_resnet)
```



Test Accuracy: 0.8032



```
=====
CLASSIFICATION REPORT
=====
```

	precision	recall	f1-score	support
Real	0.80	0.83	0.82	8061
AI-Generated	0.80	0.77	0.79	7210
accuracy			0.80	15271
macro avg	0.80	0.80	0.80	15271
weighted avg	0.80	0.80	0.80	15271

1.7 6. Model 3: EfficientNet-B2 Transfer Learning

Advanced architecture optimized for high-resolution images.

```
[41]: %%time
      # Create model
```

```

efficientnet_model = get_efficientnet_b2(num_classes=2, freeze_backbone=True).
    ↪to(device)
trainable, total = count_parameters(efficientnet_model)
print(f" EfficientNet-B2 Parameters: {trainable:,} trainable / {total:,}
    ↪total")

# Setup training
criterion_eff = nn.CrossEntropyLoss()
optimizer_eff = optim.Adam(efficientnet_model.classifier[1].parameters(),
    ↪lr=1e-4, weight_decay=1e-4)

trainer_eff = Trainer(efficientnet_model, device, criterion_eff, optimizer_eff,
    ↪use_amp=True)

# Train
print("\n Training EfficientNet-B2...")
history_eff = trainer_eff.fit(train_loader, val_loader, num_epochs=5,
                             save_path='models/efficientnet_b2_best.pth')

```

Downloading:

"https://download.pytorch.org/models/efficientnet_b2_rwrightman-c35c1473.pth" to
 /root/.cache/torch/hub/checkpoints/efficientnet_b2_rwrightman-c35c1473.pth

100%| | 35.2M/35.2M [00:00<00:00, 238MB/s]

EfficientNet-B2 Parameters: 2,818 trainable / 7,703,812 total

Training EfficientNet-B2...

Epoch 1/5

```

-----
[ 381/3818] Loss: 0.6346 | Acc: 0.6392
[ 381/3818] Loss: 0.6346 | Acc: 0.6392
[ 762/3818] Loss: 0.5988 | Acc: 0.6807
[ 762/3818] Loss: 0.5988 | Acc: 0.6807
[1143/3818] Loss: 0.5775 | Acc: 0.6999
[1143/3818] Loss: 0.5775 | Acc: 0.6999
[1524/3818] Loss: 0.5625 | Acc: 0.7108
[1524/3818] Loss: 0.5625 | Acc: 0.7108
[1905/3818] Loss: 0.5517 | Acc: 0.7177
[1905/3818] Loss: 0.5517 | Acc: 0.7177
[2286/3818] Loss: 0.5439 | Acc: 0.7231
[2286/3818] Loss: 0.5439 | Acc: 0.7231
[2667/3818] Loss: 0.5374 | Acc: 0.7280
[2667/3818] Loss: 0.5374 | Acc: 0.7280
[3048/3818] Loss: 0.5316 | Acc: 0.7328
[3048/3818] Loss: 0.5316 | Acc: 0.7328

```

[3429/3818] Loss: 0.5263 | Acc: 0.7363
[3429/3818] Loss: 0.5263 | Acc: 0.7363
[3810/3818] Loss: 0.5221 | Acc: 0.7391
[3810/3818] Loss: 0.5221 | Acc: 0.7391
Train Loss: 0.5220 | Train Acc: 0.7391
Val Loss: 0.5452 | Val Acc: 0.7188
Saved best model (val_acc: 0.7188)

Epoch 2/5

Train Loss: 0.5220 | Train Acc: 0.7391
Val Loss: 0.5452 | Val Acc: 0.7188
Saved best model (val_acc: 0.7188)

Epoch 2/5

[381/3818] Loss: 0.4820 | Acc: 0.7675
[381/3818] Loss: 0.4820 | Acc: 0.7675
[762/3818] Loss: 0.4815 | Acc: 0.7668
[762/3818] Loss: 0.4815 | Acc: 0.7668
[1143/3818] Loss: 0.4838 | Acc: 0.7651
[1143/3818] Loss: 0.4838 | Acc: 0.7651
[1524/3818] Loss: 0.4818 | Acc: 0.7658
[1524/3818] Loss: 0.4818 | Acc: 0.7658
[1905/3818] Loss: 0.4815 | Acc: 0.7658
[1905/3818] Loss: 0.4815 | Acc: 0.7658
[2286/3818] Loss: 0.4815 | Acc: 0.7666
[2286/3818] Loss: 0.4815 | Acc: 0.7666
[2667/3818] Loss: 0.4796 | Acc: 0.7677
[2667/3818] Loss: 0.4796 | Acc: 0.7677
[3048/3818] Loss: 0.4792 | Acc: 0.7678
[3048/3818] Loss: 0.4792 | Acc: 0.7678
[3429/3818] Loss: 0.4787 | Acc: 0.7683
[3429/3818] Loss: 0.4787 | Acc: 0.7683
[3810/3818] Loss: 0.4787 | Acc: 0.7683
[3810/3818] Loss: 0.4787 | Acc: 0.7683
Train Loss: 0.4788 | Train Acc: 0.7683
Val Loss: 0.5410 | Val Acc: 0.7187

Epoch 3/5

Train Loss: 0.4788 | Train Acc: 0.7683
Val Loss: 0.5410 | Val Acc: 0.7187

Epoch 3/5

[381/3818] Loss: 0.4720 | Acc: 0.7737
[381/3818] Loss: 0.4720 | Acc: 0.7737

```
[ 762/3818] Loss: 0.4739 | Acc: 0.7707
[ 762/3818] Loss: 0.4739 | Acc: 0.7707
[1143/3818] Loss: 0.4746 | Acc: 0.7697
[1143/3818] Loss: 0.4746 | Acc: 0.7697
[1524/3818] Loss: 0.4729 | Acc: 0.7701
[1524/3818] Loss: 0.4729 | Acc: 0.7701
[1905/3818] Loss: 0.4727 | Acc: 0.7705
[1905/3818] Loss: 0.4727 | Acc: 0.7705
[2286/3818] Loss: 0.4728 | Acc: 0.7705
[2286/3818] Loss: 0.4728 | Acc: 0.7705
[2667/3818] Loss: 0.4733 | Acc: 0.7697
[2667/3818] Loss: 0.4733 | Acc: 0.7697
[3048/3818] Loss: 0.4731 | Acc: 0.7699
[3048/3818] Loss: 0.4731 | Acc: 0.7699
[3429/3818] Loss: 0.4728 | Acc: 0.7703
[3429/3818] Loss: 0.4728 | Acc: 0.7703
[3810/3818] Loss: 0.4724 | Acc: 0.7705
[3810/3818] Loss: 0.4724 | Acc: 0.7705
Train Loss: 0.4724 | Train Acc: 0.7705
Val   Loss: 0.5272 | Val   Acc: 0.7307
  Saved best model (val_acc: 0.7307)
```

Epoch 4/5

```
-----
Train Loss: 0.4724 | Train Acc: 0.7705
Val   Loss: 0.5272 | Val   Acc: 0.7307
  Saved best model (val_acc: 0.7307)
```

Epoch 4/5

```
-----
[ 381/3818] Loss: 0.4699 | Acc: 0.7744
[ 381/3818] Loss: 0.4699 | Acc: 0.7744
[ 762/3818] Loss: 0.4697 | Acc: 0.7721
[ 762/3818] Loss: 0.4697 | Acc: 0.7721
[1143/3818] Loss: 0.4713 | Acc: 0.7710
[1143/3818] Loss: 0.4713 | Acc: 0.7710
[1524/3818] Loss: 0.4696 | Acc: 0.7722
[1524/3818] Loss: 0.4696 | Acc: 0.7722
[1905/3818] Loss: 0.4700 | Acc: 0.7724
[1905/3818] Loss: 0.4700 | Acc: 0.7724
[2286/3818] Loss: 0.4701 | Acc: 0.7723
[2286/3818] Loss: 0.4701 | Acc: 0.7723
[2667/3818] Loss: 0.4699 | Acc: 0.7731
[2667/3818] Loss: 0.4699 | Acc: 0.7731
[3048/3818] Loss: 0.4702 | Acc: 0.7729
[3048/3818] Loss: 0.4702 | Acc: 0.7729
[3429/3818] Loss: 0.4700 | Acc: 0.7729
[3429/3818] Loss: 0.4700 | Acc: 0.7729
```

[3810/3818] Loss: 0.4693 | Acc: 0.7733
[3810/3818] Loss: 0.4693 | Acc: 0.7733
Train Loss: 0.4694 | Train Acc: 0.7732
Val Loss: 0.5125 | Val Acc: 0.7464
Saved best model (val_acc: 0.7464)

Epoch 5/5

Train Loss: 0.4694 | Train Acc: 0.7732
Val Loss: 0.5125 | Val Acc: 0.7464
Saved best model (val_acc: 0.7464)

Epoch 5/5

[381/3818] Loss: 0.4681 | Acc: 0.7702
[381/3818] Loss: 0.4681 | Acc: 0.7702
[762/3818] Loss: 0.4676 | Acc: 0.7734
[762/3818] Loss: 0.4676 | Acc: 0.7734
[1143/3818] Loss: 0.4674 | Acc: 0.7742
[1143/3818] Loss: 0.4674 | Acc: 0.7742
[1524/3818] Loss: 0.4696 | Acc: 0.7723
[1524/3818] Loss: 0.4696 | Acc: 0.7723
[1905/3818] Loss: 0.4682 | Acc: 0.7741
[1905/3818] Loss: 0.4682 | Acc: 0.7741
[2286/3818] Loss: 0.4683 | Acc: 0.7736
[2286/3818] Loss: 0.4683 | Acc: 0.7736
[2667/3818] Loss: 0.4691 | Acc: 0.7734
[2667/3818] Loss: 0.4691 | Acc: 0.7734
[3048/3818] Loss: 0.4689 | Acc: 0.7737
[3048/3818] Loss: 0.4689 | Acc: 0.7737
[3429/3818] Loss: 0.4699 | Acc: 0.7729
[3429/3818] Loss: 0.4699 | Acc: 0.7729
[3810/3818] Loss: 0.4693 | Acc: 0.7736
[3810/3818] Loss: 0.4693 | Acc: 0.7736
Train Loss: 0.4693 | Train Acc: 0.7735
Val Loss: 0.5178 | Val Acc: 0.7406

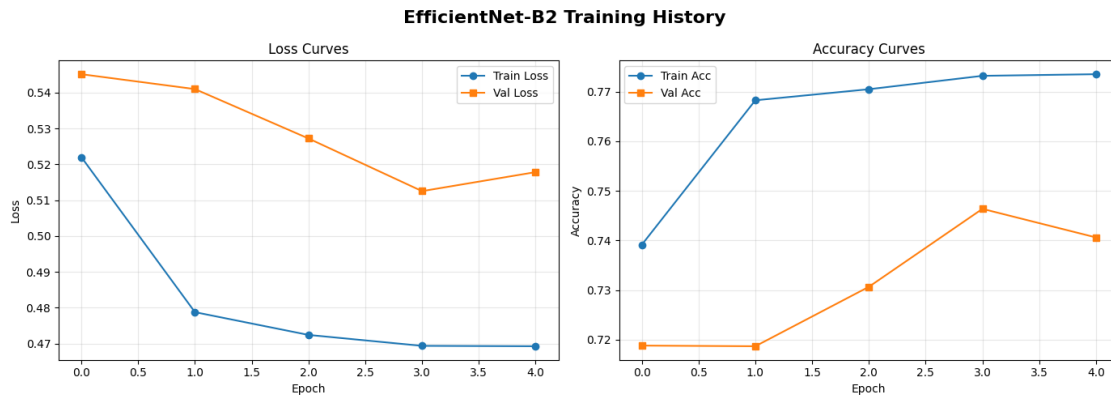
Training complete in 2821s
Best validation accuracy: 0.7464
CPU times: user 10min 11s, sys: 1min 42s, total: 11min 53s
Wall time: 47min 1s
Train Loss: 0.4693 | Train Acc: 0.7735
Val Loss: 0.5178 | Val Acc: 0.7406

Training complete in 2821s
Best validation accuracy: 0.7464
CPU times: user 10min 11s, sys: 1min 42s, total: 11min 53s
Wall time: 47min 1s

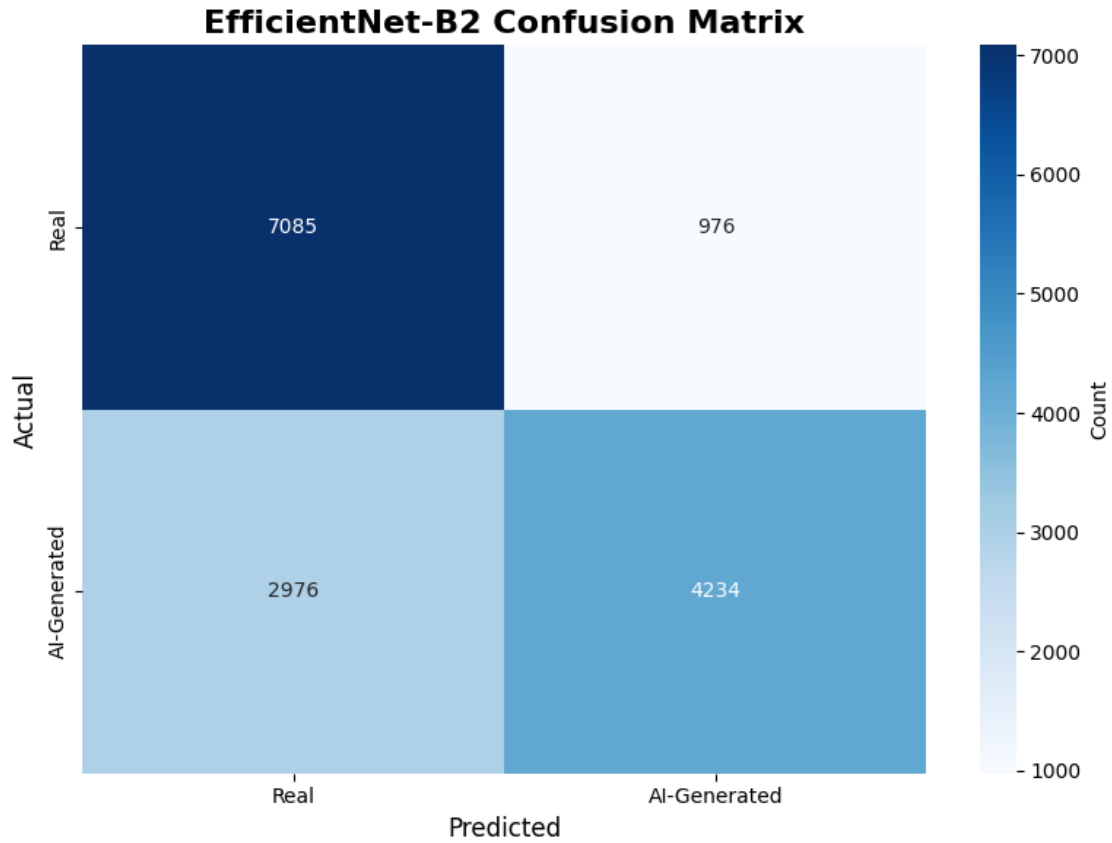
```
[42]: # Plot and evaluate
plot_training_history(history_eff, "EfficientNet-B2 Training History")

test_acc_eff, all_preds_eff, all_labels_eff = _
    ↪ evaluate_model(efficientnet_model, test_loader, device)
print(f"\n Test Accuracy: {test_acc_eff:.4f}")

plot_confusion_matrix(all_labels_eff, all_preds_eff, title='EfficientNet-B2_
    ↪ Confusion Matrix')
print_classification_report(all_labels_eff, all_preds_eff)
```



Test Accuracy: 0.7412



```
=====
CLASSIFICATION REPORT
=====
```

	precision	recall	f1-score	support
Real	0.70	0.88	0.78	8061
AI-Generated	0.81	0.59	0.68	7210
accuracy			0.74	15271
macro avg	0.76	0.73	0.73	15271
weighted avg	0.76	0.74	0.73	15271

	precision	recall	f1-score	support
Real	0.70	0.88	0.78	8061
AI-Generated	0.81	0.59	0.68	7210
accuracy			0.74	15271
macro avg	0.76	0.73	0.73	15271
weighted avg	0.76	0.74	0.73	15271

1.8 7. Model Comparison and Results

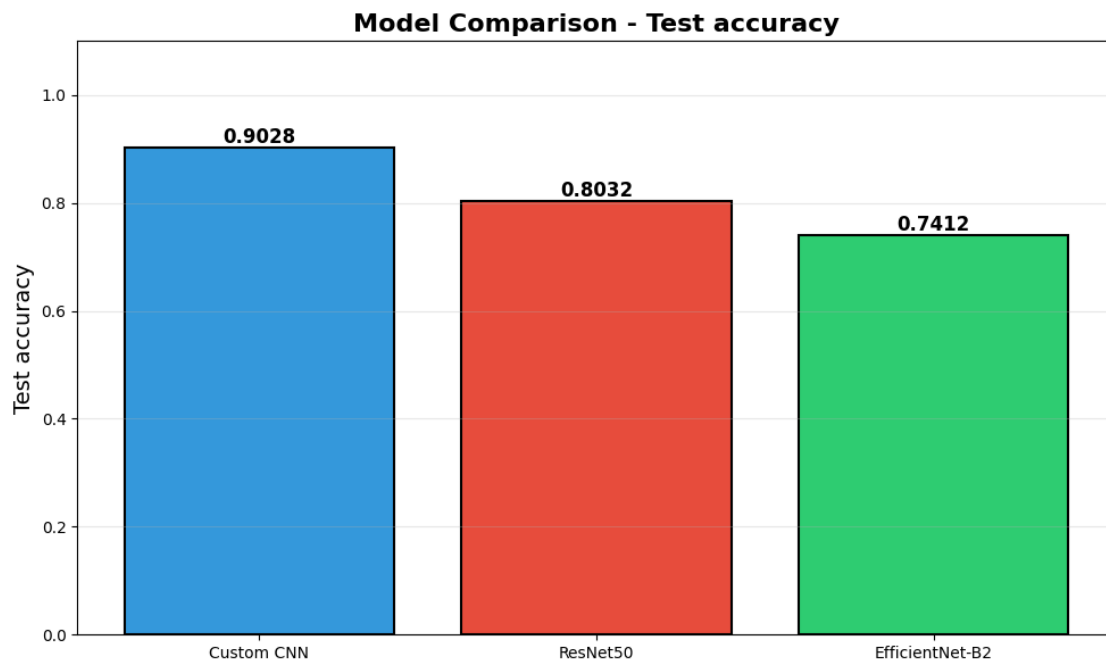
Compare all three models side-by-side.

```
[43]: # Compare test accuracies
results = {
    'Custom CNN': test_acc_cnn,
    'ResNet50': test_acc_resnet,
    'EfficientNet-B2': test_acc_eff
}

compare_models(results, metric='Test Accuracy')

print("\n" + "="*60)
print("FINAL RESULTS SUMMARY")
print("="*60)
for model_name, accuracy in results.items():
    print(f"{model_name:20s}: {accuracy:.4f} ({accuracy*100:.2f}%)")
print("="*60)

# Identify best model
best_model_name = max(results, key=results.get)
best_accuracy = results[best_model_name]
print(f"\n Best Model: {best_model_name} with {best_accuracy:.4f} accuracy")
```



=====	
FINAL RESULTS SUMMARY	
=====	
Custom CNN	: 0.9028 (90.28%)
ResNet50	: 0.8032 (80.32%)
EfficientNet-B2	: 0.7412 (74.12%)
=====	

Best Model: Custom CNN with 0.9028 accuracy

1.9 8. Conclusion and Next Steps

1.9.1 Key Findings

1. **Custom CNN:** Baseline performance with lightweight architecture
2. **ResNet50:** Strong transfer learning performance with minimal training
3. **EfficientNet-B2:** Best for capturing fine-grained details in high-resolution images

1.9.2 Model Selection Criteria

- **Speed:** Custom CNN (smallest, fastest inference)
- **Accuracy:** EfficientNet-B2 (typically best for this task)
- **Balance:** ResNet50 (good accuracy, reasonable speed)

1.9.3 Future Improvements

1. **Data Augmentation:** Add more sophisticated augmentation techniques
2. **Fine-tuning:** Unfreeze backbone layers for transfer learning models
3. **Ensemble:** Combine predictions from multiple models
4. **Attention Mechanisms:** Add attention layers to focus on synthetic artifacts
5. **Larger Dataset:** Train on more diverse AI-generated images

1.9.4 Deployment Recommendations

- **Production:** Use EfficientNet-B2 for best accuracy
- **Edge Devices:** Use Custom CNN for resource-constrained environments
- **Cloud API:** Use ResNet50 for balanced performance